

Dockeriser ses applications

Multi-stage & multi-environnement — Spring Boot, Angular, React, Python

Formateur : Haithem Mihoubi

Formation DevOps • 7 modules • 2026

Table des matières

00	Introduction : Docker & le build multi-stage	3
01	Dockeriser une application Java Spring Boot (multi-stage)	5
02	Gérer plusieurs environnements (dev / staging / prod)	8
03	Dockeriser une application Angular (multi-stage + Nginx)	12
04	Dockeriser une application React (Vite / CRA + Nginx)	15
05	Dockeriser une application Python (Flask / FastAPI / Django)	18
06	Bonnes pratiques, sécurité & checklist	21

Introduction : Docker & le build multi-stage

1. Rappels essentiels

Docker permet d'empaqueter une application **avec tout son environnement d'exécution** (runtime, dépendances, fichiers) dans une **image** portable. Un **conteneur** est une instance en cours d'exécution de cette image.

Notion	Définition courte
Image	Modèle en lecture seule, construit à partir d'un <code>Dockerfile</code> .
Conteneur	Instance exécutée d'une image (processus isolé).
Layer (couche)	Chaque instruction du Dockerfile crée une couche mise en cache.
Registry	Dépôt d'images (Docker Hub, GitLab Registry, ECR...).
Volume	Stockage persistant indépendant du cycle de vie du conteneur.

2. Le problème que résout le multi-stage

Pour compiler une application (Java, Angular, React...), on a besoin d'**outils de build** lourds : JDK + Maven, Node.js + npm, compilateurs... Mais pour **exécuter** l'application, on n'a besoin que du **résultat** (un `.jar`, des fichiers statiques).

Sans multi-stage, l'image finale contient tout : outils de build + code source + caches. Résultat : une image de **800 Mo à 1,5 Go**, plus lente à transférer et avec une surface d'attaque plus grande.

Le **build multi-stage** sépare la phase de **compilation** de la phase d'**exécution**. On compile dans une première image (« stage build »), puis on copie **uniquement l'artefact final** dans une seconde image minimale (« stage runtime »).

3. Syntaxe d'un build multi-stage

```
# ----- Stage 1 : build -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY . .
RUN mvn -B clean package -DskipTests

# ----- Stage 2 : runtime -----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/app.jar app.jar # on ne copie QUE le jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Points clés :

- `FROM ... AS build` : on **nomme** le premier stage.
- `COPY --from=build` : on copie depuis ce stage nommé, pas depuis la machine hôte.
- Tout ce qui reste dans le stage `build` (JDK, Maven, `.m2`, sources) **n'est pas** dans l'image finale.

4. Gain typique

Approche	Taille image	Surface d'attaque
Mono-stage (JDK + Maven inclus)	~750 Mo	Élevée
Multi-stage (JRE Alpine seul)	~180 Mo	Faible

5. Les 3 règles d'or de ce cours

1. **Séparer build et runtime** → image finale minimale.
2. **Optimiser le cache** → copier les fichiers de dépendances *avant* le code source.
3. **Ne jamais tourner en root** → créer un utilisateur dédié dans l'image.

Les modules suivants appliquent ces règles à Spring Boot, au multi-environnement, puis à Angular, React et Python.

Dockeriser une application Java Spring Boot (multi-stage)

1. Objectif

Construire une image Spring Boot **petite, sécurisée et rapide à reconstruire**, en utilisant un build multi-stage avec **Maven** (puis une variante **Gradle**).

2. Dockerfile complet — Maven

```
# ----- Étape 1 : build -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app

# 1) On copie d'abord le pom.xml seul → couche de dépendances réutilisable
COPY pom.xml .
RUN mvn -B dependency:go-offline          # télécharge les dépendances (mises en cache)

# 2) Puis le code source → seule cette couche change quand on code
COPY src ./src
RUN mvn -B clean package -DskipTests

# ----- Étape 2 : runtime -----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Ne pas tourner en root : on crée un utilisateur applicatif
RUN addgroup -S app && adduser -S app -G app
USER app

# Variable d'environnement par défaut (surchargée au run)
ENV SPRING_PROFILES_ACTIVE=prod

# On copie UNIQUEMENT le jar produit par le stage build
COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Pourquoi cet ordre de **COPY** ?

Docker met chaque couche en cache. Si on copie le **pom.xml** **avant** le code, alors tant que les dépendances ne changent pas, Docker **réutilise le cache** de **dependency:go-offline**. On ne re-télécharge pas Internet à chaque modification de code. C'est la règle d'or n°2 (optimisation du cache).

3. Variante Gradle

```
# ----- build -----
FROM gradle:8-jdk21 AS build
WORKDIR /app
COPY build.gradle settings.gradle ./
RUN gradle dependencies --no-daemon || true # cache des dépendances
COPY src ./src
RUN gradle bootJar --no-daemon -x test

# ----- runtime -----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
USER app
COPY --from=build /app/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

4. Optimisation avancée : les « layered jars » Spring Boot

Spring Boot peut découper le `.jar` en couches (dépendances, loader, classes de l'application). Les dépendances changent rarement → couche en cache ; seul votre code change → couche minuscule reconstruite.

```
# ----- build -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B clean package -DskipTests

# ----- extraction des couches -----
FROM eclipse-temurin:21-jre-alpine AS extract
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract

# ----- runtime -----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
USER app
COPY --from=extract /app/dependencies/ ./
COPY --from=extract /app/spring-boot-loader/ ./
COPY --from=extract /app/snapshot-dependencies/ ./
COPY --from=extract /app/application/ ./
EXPOSE 8080
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

5. Le fichier `.dockerignore` (indispensable)

Évite d'envoyer des fichiers inutiles au démon Docker (build plus rapide, image propre) :

```
target/
.git/
.idea/
*.iml
.mvn/
README.md
*.md
.env
```

6. Bonnes pratiques Spring Boot en conteneur

Bonne pratique	Pourquoi
JRE Alpine plutôt que JDK complet	Image ~5× plus petite
<code>USER app</code> (non-root)	Sécurité : limite l'impact d'une faille
Externaliser la config (<code>SPRING_PROFILES_ACTIVE</code> , env)	Une même image pour tous les environnements
Healthcheck via Actuator <code>/actuator/health</code>	Orchestrateur sait si l'app est prête
<code>-XX:MaxRAMPercentage=75</code>	La JVM respecte la limite mémoire du conteneur

Exemple de healthcheck dans `docker-compose.yml` :

```
healthcheck:
  test: ["CMD", "wget", "-q0-", "http://localhost:8080/actuator/health"]
  interval: 10s
  timeout: 3s
  retries: 5
```

7. Commandes utiles

```
docker build -t quickbite-api:1.0 .
docker run -p 8080:8080 -e SPRING_PROFILES_ACTIVE=prod quickbite-api:1.0
docker images quickbite-api          # vérifier la taille
docker history quickbite-api:1.0     # voir les couches
```

Gérer plusieurs environnements (dev / staging / prod)

1. Le principe : une image, plusieurs configurations

La règle DevOps fondamentale (« **Build once, run anywhere** ») : on construit **une seule image** et on l'adapte à chaque environnement **au démarrage**, via des **variables d'environnement** — jamais en recompilant une image par environnement.

```
[ Image unique ]  → dev      (variables dev)
                  → staging   (variables staging)
                  → prod      (variables prod)
```

2. Méthode A — Spring profiles + variables d'environnement

L'application lit son profil via `SPRING_PROFILES_ACTIVE`. On a un fichier de configuration par environnement :

```
src/main/resources/
├─ application.yml           # commun
├─ application-dev.yml       # H2, logs DEBUG
├─ application-staging.yml    # PostgreSQL de test
└─ application-prod.yml      # PostgreSQL prod, logs INFO
```

Au lancement, on choisit l'environnement **sans changer l'image** :

```
docker run -e SPRING_PROFILES_ACTIVE=dev      app:1.0
docker run -e SPRING_PROFILES_ACTIVE=staging  app:1.0
docker run -e SPRING_PROFILES_ACTIVE=prod     app:1.0
```

3. Méthode B — Fichiers `.env` + docker-compose

Docker Compose lit automatiquement un fichier `.env` à côté du `docker-compose.yml`. On crée un fichier par environnement.

`.env.dev`


```
SPRING_PROFILES_ACTIVE=dev
DATABASE_URL=jdbc:postgresql://db:5432/quickbite_dev
DATABASE_PASSWORD=devsecret
LOG_LEVEL=DEBUG
```

`.env.prod`

```
SPRING_PROFILES_ACTIVE=prod
DATABASE_URL=jdbc:postgresql://db:5432/quickbite
DATABASE_PASSWORD=${PROD_DB_PASSWORD} # injecté par le CI/CD, jamais en clair
LOG_LEVEL=INFO
```

`docker-compose.yml`

```
services:
  api:
    build: .
    ports:
      - "8080:8080"
    environment:
      SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE}
      DATABASE_URL: ${DATABASE_URL}
      DATABASE_PASSWORD: ${DATABASE_PASSWORD}
```

Lancement selon l'environnement :

```
docker compose --env-file .env.dev up -d --build
docker compose --env-file .env.prod up -d --build
```

4. Méthode C — Fichiers `compose.override` (dev vs prod)

Docker Compose fusionne automatiquement `docker-compose.yml` + `docker-compose.override.yml`. On garde le commun dans le fichier de base et les spécificités par environnement dans des fichiers séparés.

`docker-compose.yml` (commun)

```
services:
  api:
    image: quickbite-api:1.0
    ports: ["8080:8080"]
```

`docker-compose.dev.yml` (surcharge dev)

```
services:
  api:
    build: .
    environment:
      SPRING_PROFILES_ACTIVE: dev
    volumes:
      - ./src:/app/src          # hot-reload en dev
```

docker-compose.prod.yml (surcharge prod)

```
services:
  api:
    environment:
      SPRING_PROFILES_ACTIVE: prod
    restart: always
    deploy:
      resources:
        limits:
          memory: 512m
```

Lancement :

```
# Développement
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build

# Production
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

5. Méthode D — ARG au build vs ENV au runtime

Mécanisme	Quand ?	Exemple
ARG	Au build de l'image	version, URL d'un dépôt privé
ENV	Au runtime du conteneur	profil, mot de passe, URL de base de données

```
# ARG : valeur fixée au moment du build
ARG APP_VERSION=1.0.0
LABEL version=$APP_VERSION

# ENV : valeur par défaut, surchargée au docker run -e
ENV SPRING_PROFILES_ACTIVE=prod
```

```
docker build --build-arg APP_VERSION=2.3.1 -t app:2.3.1 .
docker run -e SPRING_PROFILES_ACTIVE=staging app:2.3.1
```

Règle de sécurité : un ARG ou un ENV n'est **pas** secret — il reste visible dans **docker history**. Les vrais secrets (mots de passe, clés) doivent venir d'un gestionnaire de

secrets (Docker secrets, Vault, variables CI/CD), **jamais** écrits dans le Dockerfile ou commités dans Git.

6. Tableau récapitulatif des méthodes

Méthode	Cas d'usage idéal
A — Spring profiles + <code>-e</code>	Simple, un seul conteneur
B — <code>.env</code> par environnement	Stack Compose, séparation claire dev/prod
C — <code>compose.override</code>	Comportements très différents (volumes, limites)
D — <code>ARG</code> / <code>ENV</code>	Paramétrer le build vs le runtime

Le plus courant en production : **B + D** — une image paramétrée par `ENV`, des fichiers `.env` distincts, et les secrets injectés par le pipeline CI/CD.

Dockeriser une application Angular (multi-stage + Nginx)

1. Principe

Une application Angular est **compilée** en fichiers statiques (HTML/CSS/JS) par Node.js, puis **servie** par un serveur web léger (Nginx). On a donc deux stages :

1. **build** : Node.js compile (`ng build`).
2. **runtime** : Nginx sert le dossier `dist/` — **pas de Node.js en production**.

2. Dockerfile complet

```
# ----- Étape 1 : build -----
FROM node:20-alpine AS build
WORKDIR /app

# Dépendances d'abord (cache de couche)
COPY package.json package-lock.json ./
RUN npm ci

# Puis le code, puis le build de production
COPY . .
RUN npm run build -- --configuration production

# ----- Étape 2 : runtime -----
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Angular 17+ : la sortie est dans dist/<app>/browser
COPY --from=build /app/dist/quickbite-frontend/browser /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Attention au chemin de sortie selon la version d'Angular : - Angular ≤ 16 : `dist/<nom-app>` - Angular **17+** (builder `application`) : `dist/<nom-app>/browser`

3. La configuration Nginx (`nginx.conf`)

Indispensable pour une **SPA** : toutes les routes inconnues doivent renvoyer `index.html`, sinon un rafraîchissement sur `/orders` donne une erreur 404.

```
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    # Compression
    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;
    gzip_min_length 1024;

    # Cache agressif des assets versionnés (nom avec hash)
    location ~* \.(?:js|css|woff2?|ttf|svg|png|jpg|jpeg|gif|ico)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
        try_files $uri =404;
    }

    # Routage SPA : tout le reste retombe sur index.html
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

4. Le `.dockerignore`

```
node_modules
dist
.angular
.git
Dockerfile
.dockerignore
*.md
```

5. Configurer l'URL de l'API selon l'environnement

Problème : l'URL de l'API back-end change entre dev et prod, mais on ne veut pas recompiler l'image. Solution : **injecter la configuration au démarrage** via un fichier `env.js` généré par un point d'entrée.

`docker-entrypoint.sh`

```
#!/bin/sh
# Génère un fichier JS lu par l'application au chargement
cat > /usr/share/nginx/html/assets/env.js <<EOF
window.__env = {
    apiUrl: "${API_URL:-http://localhost:8080}"
};
EOF
exec nginx -g "daemon off;"
```

Dans `index.html` : `<script src="assets/env.js"></script>` avant le bundle Angular. On lance ensuite :

```
docker run -p 8080:80 -e API_URL=https://api.prod.exemple.com quickbite-frontend
```

6. Commandes

```
docker build -t quickbite-frontend:1.0 .  
docker run -p 8080:80 quickbite-frontend:1.0  
# → http://localhost:8080
```

Image finale typique : **~50 Mo** (Nginx Alpine + fichiers statiques), contre ~1 Go si on avait gardé Node.js et `node_modules` .

Dockeriser une application React (Vite / CRA + Nginx)

1. Principe

Comme Angular, React se **compile** en fichiers statiques servis par Nginx. La seule différence est le **dossier de sortie** du build, qui dépend de l'outil :

Outil	Commande de build	Dossier de sortie
Vite	<code>npm run build</code>	<code>dist/</code>
Create React App (CRA)	<code>npm run build</code>	<code>build/</code>
Next.js (export statique)	<code>next build && next export</code>	<code>out/</code>

2. Dockerfile — React + Vite

```
# ----- Étape 1 : build -----
FROM node:20-alpine AS build
WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .
RUN npm run build          # Vite → dossier dist/

# ----- Étape 2 : runtime -----
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

3. Dockerfile — Create React App (CRA)

Seul le dossier copié change (`build` au lieu de `dist`) :

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build          # CRA → dossier build/

FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4. Configuration Nginx (routage SPA)

Identique à Angular — une SPA React (React Router) a besoin du fallback `index.html` :

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;

    location ~* \.(?:js|css|woff2?|svg|png|jpg|ico)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

5. Variables d'environnement React

⚠ Avec Vite/CRA, les variables `VITE_*` / `REACT_APP_*` sont **figées au build** (injectées dans le bundle). Deux stratégies :

Option 1 — au build (simple, une image par environnement) :

```
ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build
```

```
docker build --build-arg VITE_API_URL=https://api.prod.com -t app .
```


Option 2 — au runtime (une seule image, recommandé) : générer un `config.js` au démarrage, comme pour Angular :

```
#!/bin/sh
cat > /usr/share/nginx/html/config.js <<EOF
window.APP_CONFIG = { apiUrl: "${API_URL:-http://localhost:8080}" };
EOF
exec nginx -g "daemon off;"
```

6. `.dockerignore`

```
node_modules
dist
build
.git
Dockerfile
*.md
.env.local
```

7. Commandes

```
docker build -t react-app:1.0 .
docker run -p 3000:80 react-app:1.0
# → http://localhost:3000
```

Dockeriser une application Python (Flask / FastAPI / Django)

1. Particularité de Python

Python est **interprété** : pas de compilation en binaire. Le « build » consiste à **installer les dépendances** (`pip install`). Le multi-stage sert surtout à **séparer les dépendances de build** (compilateurs C pour certaines libs) du runtime.

2. Dockerfile — FastAPI / Flask (avec serveur de production)

```
# ----- Étape 1 : build des dépendances -----
FROM python:3.12-slim AS build
WORKDIR /app

# Outils de compilation nécessaires à certaines libs (psycpg2, etc.)
RUN apt-get update && apt-get install -y --no-install-recommends gcc \
    && rm -rf /var/lib/apt/lists/*

# Installer les dépendances dans un répertoire isolé
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ----- Étape 2 : runtime minimal -----
FROM python:3.12-slim
WORKDIR /app

# Utilisateur non-root
RUN useradd --create-home --shell /bin/bash app

# On copie UNIQUEMENT les paquets installés (pas gcc ni les caches)
COPY --from=build /install /usr/local
COPY . .

USER app
ENV PYTHONUNBUFFERED=1 \
    PORT=8000

EXPOSE 8000
# Gunicorn + Uvicorn workers pour FastAPI (production)
CMD ["gunicorn", "main:app", \
    "-k", "uvicorn.workers.UvicornWorker", \
    "--bind", "0.0.0.0:8000", "--workers", "4"]
```

Pour **Flask**, remplacer la commande finale par :

```
CMD ["unicorn", "app:app", "--bind", "0.0.0.0:8000", "--workers", "4"]
```

✗ Ne **jamais** utiliser le serveur de développement (`flask run` , `uvicorn --reload` , `python manage.py runserver`) en production : utiliser **Gunicorn** (ou Uvicorn pour l'ASGI pur).

3. Dockerfile — Django

```
# ----- build -----
FROM python:3.12-slim AS build
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends gcc libpq-dev \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ----- runtime -----
FROM python:3.12-slim
WORKDIR /app
RUN useradd --create-home app
COPY --from=build /install /usr/local
COPY . .

# Fichiers statiques collectés au build de l'image
RUN python manage.py collectstatic --noinput

USER app
ENV PYTHONUNBUFFERED=1 DJANGO_SETTINGS_MODULE=projet.settings
EXPOSE 8000
CMD ["unicorn", "projet.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "3"]
```

4. Multi-environnement avec Django/Flask

Comme pour Spring, on sépare la configuration et on choisit via une variable :

```
docker run -e DJANGO_SETTINGS_MODULE=projet.settings.prod app
docker run -e DJANGO_SETTINGS_MODULE=projet.settings.dev app
```

Ou via `.env` + Compose :

```
services:
  web:
    build: .
    environment:
      DJANGO_SETTINGS_MODULE: ${DJANGO_SETTINGS_MODULE}
      DATABASE_URL: ${DATABASE_URL}
      SECRET_KEY: ${SECRET_KEY}      # injecté par le CI/CD
```

5. Le `requirements.txt` et le cache

Copier `requirements.txt` **avant** le code permet à Docker de réutiliser la couche `pip install` tant que les dépendances ne changent pas — même logique que `pom.xml` pour Maven ou `package.json` pour npm.

6. `.dockerignore`

```
__pycache__/  
*.pyc  
.venv/  
venv/  
.git/  
.env  
*.sqlite3  
.pytest_cache/
```

7. Bonnes pratiques Python en conteneur

Bonne pratique	Pourquoi
Image <code>python:3.12-slim</code>	~10× plus petite que <code>python:3.12</code>
<code>PYTHONUNBUFFERED=1</code>	Les logs sortent immédiatement (utile en conteneur)
<code>pip install --no-cache-dir</code>	Évite de stocker le cache pip dans l'image
Serveur WSGI/ASGI (Gunicorn)	Le serveur de dev ne tient pas la charge
<code>USER app</code> non-root	Sécurité

8. Commandes

```
docker build -t mon-api-python:1.0 .  
docker run -p 8000:8000 -e PORT=8000 mon-api-python:1.0  
curl http://localhost:8000/health
```

Bonnes pratiques, sécurité & checklist

1. Récapitulatif des images de base recommandées

Technologie	Stage build	Stage runtime
Spring Boot	<code>maven:3.9-eclipse-temurin-21</code>	<code>eclipse-temurin:21-jre-alpine</code>
Angular / React	<code>node:20-alpine</code>	<code>nginx:alpine</code>
Python	<code>python:3.12-slim</code>	<code>python:3.12-slim</code>

Préférer toujours les variantes `-alpine` ou `-slim` : images minimales, moins de vulnérabilités, transfert plus rapide.

2. Les 10 règles d'or

1. **Multi-stage** : séparer build et runtime.
2. **Cache des dépendances** : copier `pom.xml` / `package.json` / `requirements.txt` avant le code source.
3. **.dockerignore** : ne pas envoyer `node_modules`, `.git`, `target`, `.env` ...
4. **Non-root** : créer et utiliser un utilisateur applicatif (`USER app`).
5. **Tags précis** : `node:20-alpine`, jamais `node:latest` (non reproductible).
6. **Une seule responsabilité** par conteneur (un process principal).
7. **Configuration par variables d'environnement**, pas en dur dans l'image.
8. **Secrets hors de l'image** : jamais de mot de passe dans le Dockerfile ou Git.
9. **Healthcheck** : permettre à l'orchestrateur de connaître l'état de l'app.
10. **Versionner les images** : `app:1.4.2`, pas seulement `latest`.

3. Sécurité : ce qu'il ne faut jamais faire

```
# ❌ MAUVAIS
FROM node:latest                # tag flottant, non reproductible
ENV DB_PASSWORD=Sup3rSecret     # secret en clair dans l'image !
USER root                       # tourne en root
COPY . .                        # copie tout, y compris .env et .git
```

```
# ✅ BON
FROM node:20-alpine
RUN addgroup -S app && adduser -S app -G app
USER app
COPY --chown=app:app . .
# Le secret vient du runtime : docker run -e DB_PASSWORD=... (ou Docker secret)
```

4. Healthchecks par technologie

```
# Spring Boot (Actuator)
healthcheck:
  test: ["CMD", "wget", "-q0-", "http://localhost:8080/actuator/health"]
  interval: 10s
  timeout: 3s
  retries: 5

# Nginx (Angular / React)
healthcheck:
  test: ["CMD", "wget", "-q0-", "http://localhost:80/"]
  interval: 10s
  retries: 3

# Python (endpoint /health applicatif)
healthcheck:
  test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8080/health')"]
  interval: 10s
  retries: 3
```

5. Réduire la taille des images

Technique	Gain
Multi-stage	-70 % à -90 %
Image alpine / slim	-50 % à -80 %
.dockerignore complet	Évite des dizaines de Mo inutiles
--no-cache-dir (pip), npm ci --omit=dev	Pas de caches dans l'image
Fusionner les RUN apt-get + nettoyer /var/lib/apt/lists	-10 à -50 Mo

6. Commandes de diagnostic

```
docker images          # tailles des images
docker history mon-image:1.0 # poids de chaque couche
docker scout cves mon-image:1.0 # scan de vulnérabilités (si Docker Scout)
docker exec -it conteneur sh # ouvrir un shell dans le conteneur
docker stats           # CPU / mémoire en temps réel
docker compose logs -f api # suivre les logs d'un service
```

7. Checklist finale avant la production

- [] Build **multi-stage** (image finale sans outils de build)
- [] Image de base **alpine/slim** avec un **tag précis**
- [] **.dockerignore** présent et complet

- [] Conteneur en **utilisateur non-root**
- [] Aucune valeur secrète dans le Dockerfile ni dans Git
- [] Configuration injectée par **variables d'environnement**
- [] **Healthcheck** défini
- [] Image **scannée** (vulnérabilités) et **taggée par version**
- [] Limites **mémoire/CPU** définies en production
- [] `docker compose config` valide et stack testée (`up -d --build`)

8. Pour aller plus loin

- **Docker Compose** : orchestrer plusieurs conteneurs (front + API + DB).
- **CI/CD** : construire et pousser l'image automatiquement (GitLab CI, GitHub Actions).
- **Kubernetes** : déployer et mettre à l'échelle les images en production.
- **Registry privé** : héberger ses images (Docker Hub, GitLab, AWS ECR).

Ces sujets prolongent naturellement ce cours : une fois l'application correctement dockerisée, l'étape suivante est son déploiement automatisé et son orchestration.